LangChain Structured Output — Complete Beginner to Advanced Explanation

If you want to learn **LangChain** properly, **Structured Output** is one of the most important concepts you must understand. Without it, building **AI applications, AI agents, RAG systems, tool-calling workflows, and APIs** becomes much more difficult.

Let's understand this concept step by step, from beginner to advanced.

A Real-World Example

Imagine you are a teacher.

You ask a student:

> **What is your name?**

The student replies:

> **My name is Sakhawat Hossen. I study ICT at Comilla University. My CGPA is 3.85.**

This answer is very easy for a human to understand.

But what about a computer?

A computer does not automatically know:

- Which part is the name?
- Which part is the university?
- Which part is the CGPA?

To the computer, it is simply one large block of text.

This is called **Unstructured Output**.

"My name is Sakhawat Hossen. I study ICT at Comilla University. My CGPA is 3.85."

Everything is mixed together.

Now Imagine the AI Returns This

```
{
  "name": "Sakhawat Hossen",
  "university": "Comilla University",
  "department": "ICT",
  "cgpa": 3.85
}
```

Now the computer immediately understands:

- where the name is
- where the university is
- where the CGPA is

This is called **Structured Output**.

What Does "Structured" Mean?

Structured means the data follows a predefined format.

Think about a school admission form.

Name:
Age:
Class:
Roll:

Here:

- The Name goes in the Name field.
- The Age goes in the Age field.

You cannot write your age where your name should be.

That is a **structured format**.

Now imagine someone simply says:

> Write something about yourself.

One person may write five lines.

Another may write ten lines.

Someone else may forget to mention their name.

This is **Unstructured Output** because there is no fixed format.

Why Do We Need Structured Output?

Suppose you are building an **AI Resume Parser**.

A user uploads this resume:

Sakhawat Hossen

Phone: 017xxxxxxxx

Email:
abc@gmail.com

Skills

Python
Machine Learning
Deep Learning

If the AI returns:

He knows Python and Machine Learning.

How will your database store it?

Where is the phone number?

Where is the email?

Where are all the skills?

The program cannot reliably extract those values.

Instead, if the AI returns:

```
{
  "name": "Sakhawat Hossen",
  "phone": "017xxxxxxxx",
  "email": "abc@gmail.com",
  "skills": [
    "Python",
    "Machine Learning",
    "Deep Learning"
  ]
}
```

Now inserting the information into a database becomes extremely easy.

Another Example

Suppose an Amazon review says:

This laptop is amazing.

Battery is excellent.

Display is beautiful.

But speaker quality is poor.

You ask the AI to analyze the review.

Unstructured Output

Overall this laptop is very good.

This summary is useful for humans, but a program cannot easily process it.

Structured Output

```
{
  "topic": "Laptop",
  "pros": [
```

```
    "Battery",
    "Display"
  ],
  "cons": [
    "Speaker"
  ],
  "sentiment": "Positive"
}
```

Now this output can be directly used in:

- Websites
- Dashboards
- Analytics systems
- APIs

Another Example

Suppose you are building a food delivery application.

The user writes:

I want two burgers and one coke.

If the AI replies:

Sure, I'll order that.

Can the program understand exactly what to order?

No.

Instead:

```
{
  "burger": 2,
  "coke": 1
}
```

Now this data can be directly sent to an ordering API.

Why Doesn't an LLM Produce Structured Output by Default?

Large Language Models are designed to generate **natural language**.

For example:

Prompt:

Explain Machine Learning.

The model naturally generates:

Machine Learning is a branch of Artificial Intelligence...

It produces paragraphs because it is a **Language Model**, not a database.

So What Does LangChain Do?

LangChain tells the model:

> Don't respond with a paragraph.
>
> Respond using this specific format.

That is the idea behind **Structured Output**.

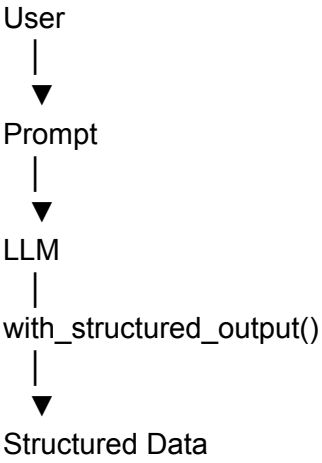
How Does LangChain Do This?

LangChain provides a function called:

with_structured_output()

This function wraps the language model and forces it to follow a predefined structure.

Workflow:



Example

Normal Output

Rahim
Age 20
Student

Structured Output

```
{
  "name": "Rahim",
  "age": 20,
  "occupation": "Student"
}
```

How Does LangChain Know Which Fields to Return?

You must define the structure yourself.

For example:

- Name
- Age
- Department

LangChain then instructs the LLM to produce exactly these fields.

There are three common ways to define this structure.

Method 1 — TypedDict

This is the simplest approach.

```
class Student(TypedDict):
    name: str
    age: int
    cgpa: float
```

This tells the model:

Return these three fields.

Example output:

```
{
  "name": "Rahim",
  "age": 20,
  "cgpa": 3.7
}
```

Done.

The Problem

Suppose the AI returns:

```
{
  "name": "Rahim",
  "age": "Twenty"
}
```

TypedDict will not complain.

Or:

cgpa = 100

Again, no error.

TypedDict only describes the expected structure.

It does **not validate** the data.

What Is Validation?

Validation means checking whether the data is correct.

Correct:

Age = 20

Incorrect:

Age = Apple

Validation detects invalid data and raises an error.

Method 2 — Pydantic

Pydantic is the most powerful and commonly used approach.

Suppose you define:

Name → String

Age → Integer

CGPA → Float

If the AI returns:

Age = Twenty

Pydantic immediately reports:

> Age must be an integer.

It raises an error.

Another Example

You define:

CGPA must be between 0 and 4

The AI returns:

CGPA = 5.5

Pydantic reports:

Validation Error

This prevents invalid data from entering your application.

What Is Type Coercion?

Type Coercion is another useful feature of Pydantic.

Suppose the AI returns:

"20"

This is a string.

But your program expects an integer.

Pydantic automatically converts:

"20"

↓

20

Similarly,

"3.75"

↓

3.75

It automatically converts compatible values into the correct type.

What Is Field()?

`Field()` is used to provide descriptions, constraints, and validation rules.

Example:

```
age: int = Field(description="Student age")
```

or

```
cgpa: float = Field(
    ge=0,
    le=4,
    description="CGPA must be between 0 and 4"
)
```

This helps both:

- the language model understand the expected value
- Pydantic validate the data

What Is JSON Schema?

Suppose your system has:

Backend → Python

Frontend → JavaScript

Database → MongoDB

Every component needs the same data format.

TypedDict is Python-specific.

Pydantic is also Python-specific.

In this situation, we use **JSON Schema** because it is language-independent.

Example:

```
{
  "type": "object",
  "properties": {
    "name": {
      "type": "string"
    },
```

```
    "age": {
      "type": "integer"
    }
  }
}
```

Python, JavaScript, Java, Go, and many other languages understand JSON Schema.

Two Ways to Receive Structured Output

1. JSON Mode

The model simply returns JSON.

Example:

```
{
  "name": "Rahim"
}
```

Nothing more.

---

2. Function Calling

Function Calling is mainly used for AI Agents.

Suppose you have a calculator function:

add()

Instead of generating:

2 + 5 = 7

The model returns structured arguments:

Function: add

Arguments:
2
5

The AI Agent can directly execute the function.

Why Is Function Calling Important?

AI Agent Workflow

User

↓

Calculate 25 × 15

↓

LLM

↓

Tool Calling

↓

Calculator

↓

375

↓

LLM

↓

Final Answer

If the LLM only generated plain text, the calculator could not understand what to execute.

That is why Function Calling is essential.

What Is an Output Parser?

Not every model supports Structured Output.

Many smaller open-source models generate plain text.

Example:

Name Rahim

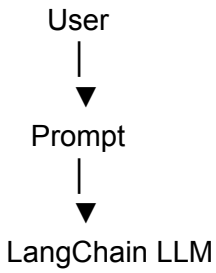
Age 20

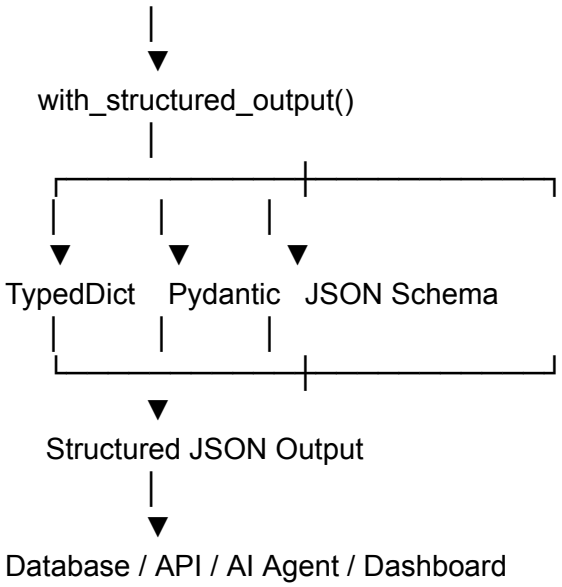
An Output Parser converts this into:

```
{
  "name": "Rahim",
  "age": 20
}
```

Its job is to transform unstructured text into structured data.

Complete Workflow





When Should You Use Each Method?

Situation	Recommended Choice	Reason
Quick prototype	TypedDict	Simple and requires minimal code
Production application	Pydantic	Validation, type conversion, and constraints
Multi-language project	JSON Schema	Supported across many programming languages
Only JSON output is needed	JSON Mode	Returns JSON directly
AI Agents or Tool Calling	Function Calling	Best for passing structured arguments to tools

---

Final Summary

Large Language Models naturally generate **human-readable text**, but real-world software systems require **machine-readable structured data**.

LangChain solves this problem through `with_structured_output()`, allowing developers to define an output schema using **TypedDict**, **Pydantic**, or **JSON Schema**.

The LLM then attempts to generate responses that match the predefined schema, making its output reliable and immediately usable in databases, APIs, AI agents, dashboards, and other software systems.

In short, **Structured Output acts as the bridge between natural language generated by LLMs and the structured data required by modern software applications.**